

source solutions and external services—a few for internal components and some for enriched integration support with inbound and outbound event processing.

- MongoDB as the default data store
- Mosquitto as the MQTT broker (even though HiveMQ is mentioned in the documentation)
- ActiveMQ, RabbitMQ for rich protocol support like AMQP, STOMP, etc.
- Eclipse Californium for CoAP messaging
- InfluxDB for time series data and Grafana for visualisation
- HBase for distributed clustering support for device events
- Hazelcast, an in-memory data grid for external event processing
- Mulesoft Any Point Platform for ESB functionality
- Azure Event Hub and Amazon SQS client libraries for event forwarding
- WSO2 Siddhi for Complex Event Processing (CEP)
- WSO2 Identity Server for asset management
- Apache Spark for streaming analytics
- Apache Solr for indexing and searching device events
- Twilio client for SMS alerts over the cloud
- Mapping devices with Magento products as assets through the community edition

It can also communicate with external cloud services like *dweet.io* for external messaging and InitialState for advanced data visualisation. It comes with an add-on for OpenHAB persistence support and for controlling back devices via outbound events.

Installation

Installation of SiteWhere is very simple, with few dependencies. Install OpenJDK 1.8, MongoDB and any MQTT broker like HiveMQ or Mosquitto as dependencies. Start MongoDB and MQTT broker. Download the tarball of the latest stable release (v1.9.0 at present), extract it to a suitable location, say */opt*, then switch to the extracted directory, i.e., */opt/sitewhere-server-1.9.0* and execute the script *bin/startup.sh*. To customise the server port, edit *conf/application.properties* and uncomment the line *server.port=8080*.

Building from sources

If you want to build SiteWhere from sources, check out the latest branch, say *SiteWhere-1.10.0* or *master* from github.com/sitewhere/sitewhere.git and run the following commands.

To create a Web application archive (WAR) file, which can be deployed on a Tomcat-like server, type:

```
gradle clean war
```

To generate a standalone archive with an embedded server, similar to the downloaded tarball, use the following command:

```
gradle clean serverTar
```

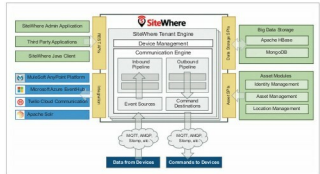


Figure 1: Architecture and integration of SiteWhere

Collect the generated WAR file or tar file from the *build/distributions* sub-directory.

Running SiteWhere in a Docker container

You can even run SiteWhere in a Docker container, for which you can check out the source code of any recent version, navigate to *sitewhere-docker/standalone/1.8.0*, and replace the Ubuntu version 14.04 with 16.04 in Dockerfile for better package support. Run the following command to build the image:

```
docker build -t sitewhere/standalone:1.8.0 .
```

To launch the container for the built image, use the following command:

```
docker run -p 8080:8080 -p 1883:1883 -p 61623:61623 \
--name mysitewhere sitewhere/standalone:1.8.0
```

To customise the port, launch the shell using the following command, switch to */opt/sitewhere/conf* and edit *application.properties* as mentioned above:

```
docker exec -it mysitewhere /bin/bash
```

If SiteWhere doesn't start manually, launch the server as follows:

```
docker exec -it mysitewhere /opt/sitewhere/bin/startup.sh
```

SiteWhere can also be deployed on cloud environments like Amazon EC2 and Microsoft Azure for public access.

Note: There is a problem in downloading images from specified repositories in the Docker hub. I suggest you build your own from available Docker script.

The first steps

You can access the HTML5 based *Admin* interface on the URL path */sitewhere/admin* of the chosen host and port. An interactive REST interface based on Swagger is available on the path */sitewhere*. For example, if the server is installed on